

D	S	T	Q	Q	S	S
D	L	M	M	J	V	S

Tempo de prova na folha: 1 hora e 10 min
 Tempo de prova VS: 50 minutos

Pronta 2025.2

A.

0,8 ✓

```
public static int union(int[] a, int[] b, int[] u) {  
    int tamU = 0;
```

```
    for (int i = 0; i < tamA; i++) {  
        if (!existe(u, tamU, a[i])) {  
            u[tamU] = a[i];  
            tamU++;  
        }  
    }
```

< tamB

```
    for (int i = 0; i <= tamB; i++) {  
        if (!existe(u, tamU, b[i])) {  
            u[tamU] = b[i];
```

*Faltou adicionar
tamU++*

```
    }  
    return tamU;
```

B.

0.8 ✓

```

public static void order(int[] v, int n) {
    for (int i = 1; i < n; i++) {
        int chare = v[i];
        int j = i;
        // correction i-1;
        while (j > 0 && v[j] > chare) {
            v[j+1] = v[j];
            j--;
        }
        // v[j+1]
        v[j+1] = chare;
    }
}
    
```


C

0,9

```
public static int getVelocity(int[] v, int fromV,  
int[] vsr) {
```

```
    int fromVSR = 0;
```

Fallout!

```
    for (int i = 0; i < fromV; i++) {
```

```
        if (exists(vsr, fromVSR, v[i])) {
```

```
            vsr[fromVSR] = v[i];
```

```
            fromVSR++;
```

```
        }
```

```
    }
```

```
    return fromVSR;
```

```
}
```

D.

0,7

```
Public static void rotateMatrix(int[] v, int tam,
int k) {
```

```
int[] temp = new int[tam];
```

← Para ser feito in-place

```
for(int i = 0; i < tam; i++) {
```

```
temp[i] = v[(i+k) % tam];
```

```
}
```

```
v = temp;
```

```
}
```

Obs: Faltou k negativo

← ~~erro~~

Tempo: 1 hora e 10 minutos